Circuitos Combinatórios

João Canas Ferreira

Outubro de 2026

U.PORTO
FEUP FACULDADE DE ENGENHARIA
UNIVERSIDADE DO PORTO

Tópicos

1. Álgebra de Boole
   - Representação Abstrata do Processamento Binário
   - Especificação Algébrica
   - Representações Canónicas
   - Mapas de Karnaugh
2. Portas Lógicas
   - Portas Lógicas Elementares
   - Descrição Hierárquica de Circuitos
3. Circuitos Padrão
   - Multiplexadores
   - Descodificadores Binários
   - Codificadores

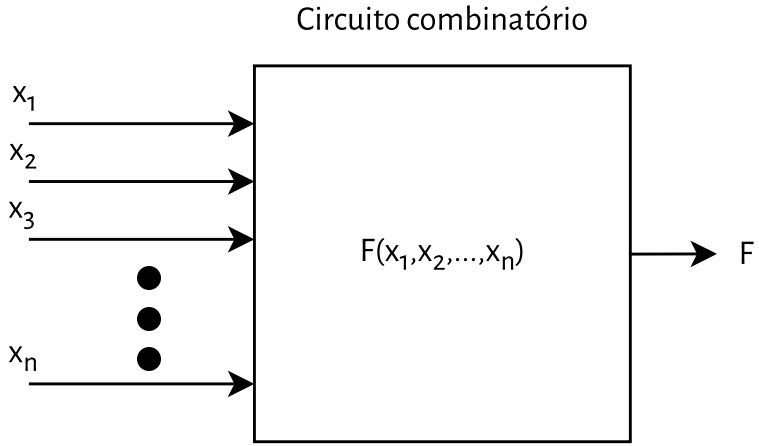
1 Álgebra de Boole
- Representação Abstrata do Processamento Binário
- Especificação Algébrica
- Representações Canónicas
- Mapas de Karnaugh

2 Portas Lógicas
- Portas Lógicas Elementares
- Descrição Hierárquica de Circuitos

3 Circuitos Padrão
- Multiplexadores
- Descodificadores Binários
- Codificadores

Tratamento de Informação Binária

➠ Como definir e representar o tratamento de informação binária?



➠ Modelo concetual mais simples: "caixa negra" que tem *n* entradas e 1 saída. O valor binário da saída depende da **combinação** de valores binários presentes nas entradas.

➠ Como definir a função *F* das *n* entradas binárias?

Definição Exaustiva (tabular)

➠ Como as combinações de valores de entrada são finitas, podemos fazer uma lista (tabela) exaustiva do valor de saída correspondente a cada uma. Exemplo:

x_2	x_1	x_0	F
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

➠ Tabela de verdade (considerando 1 como verdadeiro e 0 como falso)

➠ Para uma função de n variáveis binárias, quantas linhas tem a tabela?

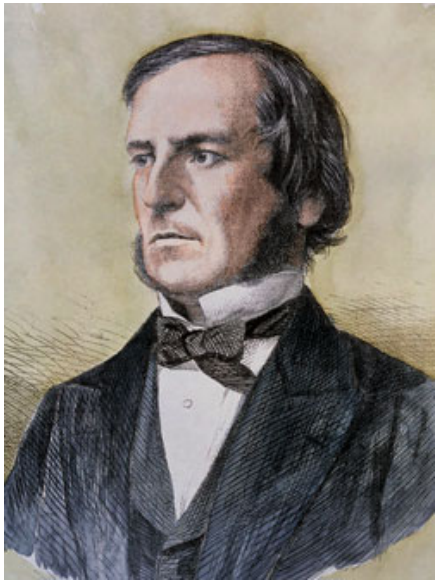
➠ Como calcular a **composição de funções**? (Quando um valor de entrada é, por sua vez, função de outros valores?)

Expressões Algébricas

➠ A especificação e composição das funções de variáveis binárias pode ser simplificada com a introdução de expressões algébricas.

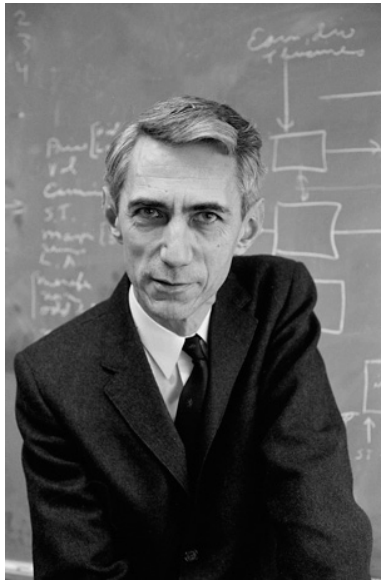
➠ Em 1938, Claude Shannon propôs a utilização de um método de cálculo inventado no século XIX (1854) pelo Reverendo George Boole para expressar as "leis do raciocínio".
Nota: Esse método é equivalente ao cálculo da lógica proposicional.

➠ A formalização das operações associadas designa-se por **Álgebra de Boole** e pode ser feita sem requerer uma interpretação como "leis do raciocínio".



George Boole

Claude Shannon



Axiomas de Álgebra de Boole

➠ Uma **álgebra de Boole** é constituída por um conjunto A, dotado de duas operações binárias $+$ e $\bullet$, uma operação unária $\overline{}$ (complemento) e tendo (pelo menos) dois elementos distintos 0 e 1.

➠ Para quaisquer $x, y, z \in A$, valem os seguintes axiomas (Huntington, 1904)

$x + 0 = x$	$x \cdot 1 = x$	(identidade)
$x + y = y + x$	$x \cdot y = y \cdot x$	(comutatividade)
$x + (y \cdot z) = (x + y) \cdot (x + z)$	$x \cdot (y + z) = (x \cdot y) + (x \cdot z)$	(distributividade)
$x + \overline{x} = 1$	$x \cdot \overline{x} = 0$	(complemento)

➠ Notações alternativas: $\vee$ para $+$ $\wedge$ para $\bullet$ $\neg x$ para $\overline{x}$.

➠ **Princípio da dualidade:** A uma igualdade verdadeira corresponde outra igualdade verdadeira obtida pelas trocas seguintes:

$$+ \leftrightarrow \bullet \qquad\qquad \mathbf{0} \leftrightarrow \mathbf{1}$$

(Porquê?)

➠ Para circuito digitais:

álgebra de Boole com $A = \{0, 1\}$ (apenas dois elementos: 0 e 1)

Alguns Teoremas Úteis

➠ Precedência decrescente: negação, e-lógico (•), ou-lógico (+).

1 variável	
$x + x = x$	$x \cdot x = x$
$x + 1 = 1$	$x \cdot 0 = 0$
$\overline{\overline{x}} = x$	
2 variáveis	
$x + x \cdot y = x$	$x \cdot (x + y) = x$
$x + \overline{x} \cdot y = x + y$	$x \cdot (\overline{x} + y) = x \cdot y$
$(x + y) + (\overline{x} \cdot \overline{y}) = 1$	$(x \cdot y) \cdot (\overline{x} + \overline{y}) = 0$
$(x + y) \cdot (\overline{x} \cdot \overline{y}) = 0$	$(x \cdot y) + (\overline{x} + \overline{y}) = 1$
$\overline{x + y} = \overline{x} \cdot \overline{y}$	$\overline{x \cdot y} = \overline{x} + \overline{y}$
3 variáveis	
$x \cdot (y \cdot z) = (x \cdot y) \cdot z$	$x + (y + z) = (x + y) + z$
$x \cdot y + \overline{x} \cdot z + y \cdot z = x \cdot y + \overline{x} \cdot z$	$(x + y) \cdot (\overline{x} + z) \cdot (y + z) = (x + y) \cdot (\overline{x} + z)$

leis de de Morgan (não é gralha!)

➠ Para a álgebra de Boole (de 2 elementos), os teoremas podem ser demonstrados construindo as tabelas de verdade das expressões de ambos os lados da igualdade e confirmando que as colunas dos resultados são iguais.

Simplificação de Expressões

- Axiomas e teoremas podem ser usados na simplificação de expressões.
  Exemplo (por convenção, pode omitir-se o operador $\bullet$):

$$
\begin{aligned}
\overline{AB}\,\left(\overline{A}+B\right)\left(\overline{B}+B\right) &= \overline{AB}\,\left(\overline{A}+B\right) \\
&= \left(\overline{A}+\overline{B}\right)\,\left(\overline{A}+B\right) && \text{lei de de Morgan} \\
&= \overline{A}+\overline{B}B && \text{distributividade} \\
&= \overline{A}
\end{aligned}
$$

- Alternativas:
  - mapas de Karnaugh (método gráfico; até 6 variáveis)
  - método Quine-McClusky; determina expressões mínimas de dois níveis
  - métodos heurísticos:
    técnicas que tentam encontrar uma solução boa ou suficientemente próxima da ideal de forma eficiente, sem garantir que seja a melhor solução possível.

Teorema de Expansão de Boole (Shannon)

➠ Para qualquer função booleana vale:

$$F(x_1, x_2, \ldots, x_n) = x_1 \cdot F(1, x_2, \ldots, x_n) + \overline{x_1} \cdot F(0, x_2, \ldots, x_n)$$

➠ A aplicação repetida da expansão permite escrever qualquer função na **forma canónica disjuntiva** (soma de produtos).

Exemplo para duas variáveis:

$$\begin{aligned} F(x_1, x_2) &= x_1 \cdot F(1, x_2) + \overline{x_1} \cdot F(0, x_2) \\ &= x_1 x_2 \cdot F(1,1) + x_1 \overline{x_2} \cdot F(1,0) + \overline{x_1} x_2 \cdot F(0,1) + \overline{x_1}\,\overline{x_2} \cdot F(0,0) \end{aligned}$$

A expressão corresponde à tabela de verdade:

x_1	x_2	$F(x_1, x_2)$
0	0	$F(0,0)$
0	1	$F(0,1)$
1	0	$F(1,0)$
1	1	$F(1,1)$

Forma Canónica Disjuntiva

- Uma função booleana de n variáveis pode ser expressa por uma soma de produtos (termos), em que cada produto inclui **uma e uma só vez cada uma das variáveis ou o seu complemento**.

x_2	x_1	x_0	F
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

$$F(x_2, x_1, x_0) = (\overline{x_2}\,\overline{x_1}\,\overline{x_0}) + (\overline{x_2}x_1x_0) + (x_2\overline{x_1}\,\overline{x_0}) + (x_2x_1\overline{x_0})$$

- Cada um destes termos designa-se por **termo mínimo** ou *minterm*.

- Existe uma *correspondência direta* entre a forma canónica disjuntiva de uma função booleana e a sua tabela de verdade (variáveis na mesma ordem).

Somas de Produtos Mínimas

- A forma canónica disjuntiva mostra que é possível representar qualquer função booleana com **expressões de dois níveis**.
- A forma canónica disjuntiva é uma soma de produtos (SOP), mas não é, geralmente, a expressão desse tipo com o menor número de termos ou os termos mais simples (com menos variáveis).
- Mapas de Karnaugh (ou o método de Quine-McCluskey) permitem obter SOPs mínimas de forma sistemática.
- Também se pode usar simplificação algébrica.

$$
\begin{aligned}
F(x_2, x_1, x_0) &= (\overline{x_2}\,\overline{x_1}\,\overline{x_0}) + (\overline{x_2}x_1x_0) + (x_2\overline{x_1}\,\overline{x_0}) + (x_2x_1\overline{x_0}) \\
&= (\overline{x_2}\,\overline{x_1}\,\overline{x_0}) + (\overline{x_2}x_1x_0) + x_2\overline{x_0}(\overline{x_1} + x_1) \\
&= (\overline{x_2}\,\overline{x_1}\,\overline{x_0}) + (\overline{x_2}x_1x_0) + x_2\overline{x_0}
\end{aligned}
$$

Forma Canónica Conjuntiva

- Devido ao princípio da dualidade, uma função booleana de n variáveis também pode ser expressa por um produto de somas , em que cada soma inclui **uma e uma só vez cada uma das variáveis ou o seu complemento**.

x_2	x_1	x_0	F
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

$$F(x_2, x_1, x_0) = (x_2 + x_1 + \overline{x_0}) \cdot (x_2 + \overline{x_1} + x_0) \cdot (\overline{x_2} + x_1 + \overline{x_0}) \cdot (\overline{x_2} + \overline{x_1} + \overline{x_0})$$

- Cada um destes termos designa-se por **termo máximo** ou *maxterm*.
- A forma canónica conjuntiva é um **produto de somas** (POS), mas não é, geralmente, a expressão mais simples desse tipo.
- Correspondência direta entre a forma canónica conjuntiva de uma função booleana e a sua tabela de verdade (variáveis na mesma ordem).

Mapas de Karnaugh: conceitos

Teorema da adjacência lógica

Dois termos mínimos que difiram em apenas uma única variável podem ser combinados, eliminando a variável que aparece negada e não-negada.

$$A \cdot B + A \cdot \overline{B} = A \cdot (B + \overline{B}) = A$$

Mapas (ou quadros) de Karnaugh:

- Cada célula corresponde a um termo mínimo.
- Células adjacentes correspondem a termos que diferem apenas numa variável.

Código Gray

Um código em que elementos adjacentes variam em apenas um dígito.

00

01

11

10

Mapa de Karnaugh para funções de 3 variáveis

Mapas (ou quadros) de Karnaugh:

- Cada célula corresponde a um termo mínimo.
- Células adjacentes correspondem a termos que diferem apenas numa variável.
- Linhas e colunas identificadas por códigos Gray

Exemplo para uma função de três variáveis $F(A, B, C)$:

C \ BA	00	01	11	10
0	$\overline{A}\,\overline{B}\,\overline{C}$	$A\,\overline{B}\,\overline{C}$	$A\,B\,\overline{C}$	$\overline{A}\,B\,\overline{C}$
1	$\overline{A}\,\overline{B}\,C$	$A\,\overline{B}\,C$	$A\,B\,C$	$\overline{A}\,B\,C$

Mapa de Karnaugh para funções de 3 variáveis

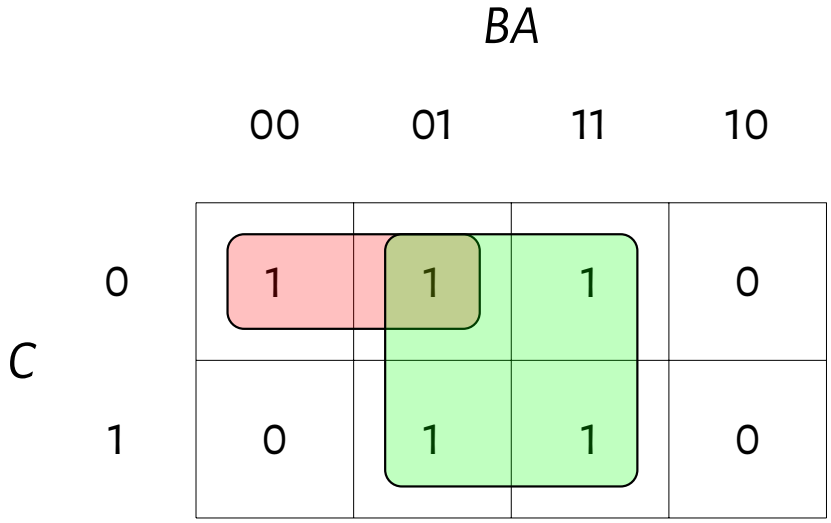
Exemplo para a função de três variáveis $F(A, B, C)$ dada pela tabela de verdade:

C	B	A	F
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

C \ BA	00	01	11	10
0	1	1	1	0
1	0	1	1	0

Expressão SOP mínima: formar grupos de uns

- Grupos devem ser retângulos de células adjacentes.
- Número de células do grupo é uma potência de 2.
- Criar os **maiores** grupos possíveis.
- Cobrir todos os uns com o **menor número total** de grupos.



Grupo vermelho P_1:
Valores C (0) e B (0) das «coordenadas» são comuns a todas as suas células.
A expressão $\mathbf{G_1} = \overline{\mathbf{C}} \cdot \overline{\mathbf{B}}$ é **1** sse $C = 0 \wedge B = 0$

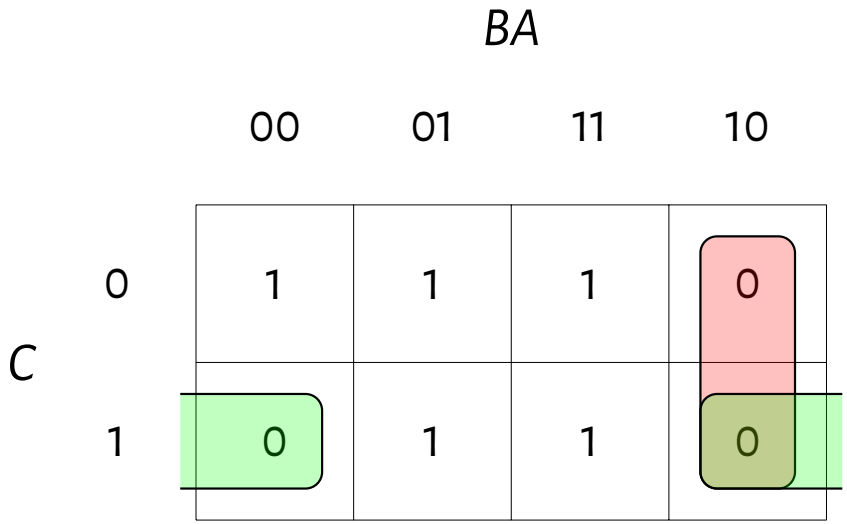
Grupo verde P_2:
Valor A (1) das «coordenadas» é comum a todas suas as células.
A expressão $\mathbf{G_2} = \mathbf{A}$ é **1** sse $A = 1$.

A função $F(A, B, C)$ é **1** se e só se (sse) $G_1 = 1$ ou $G_2 = 1$. Logo,

$$F(A, B, C) = P_1 + P_2 = \overline{C} \cdot \overline{B} + A$$

Expressão POS mínima: formar grupos de zeros

- Usar conceitos análogos para grupos de zeros.



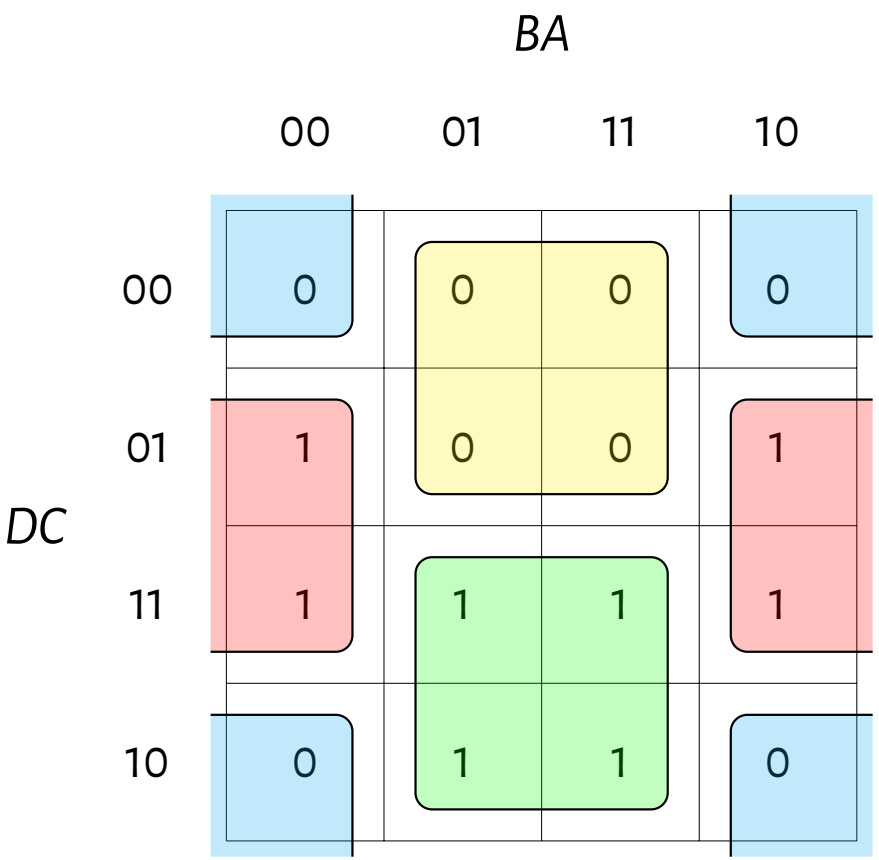
Grupo vermelho G_1:
Valores B (1) e A (0) das «coordenadas» são comuns a todas as suas células.
A expressão $\mathbf{S_1} = \overline{\mathbf{B}} + \mathbf{A}$ é **0** sse $B = 1 \wedge A = 0$

Grupo verde S_2:
Valores C (1) e A (0) das «coordenadas» são comuns todas as suas células.
A expressão $\mathbf{G_2} = \overline{\mathbf{C}} + \mathbf{A}$ é **0** sse $C = 1 \wedge A = 0$.

A função $F(A, B, C)$ é **0** se e só se (sse) $G_1 = 0$ e $G_2 = 0$. Logo,

$$F(A, B, C) = S_1 \cdot S_2 = (\overline{B} + A) \cdot (\overline{C} + A)$$

Mapas para funções de 4 variáveis



Soma de produtos:

- Grupo vermelho: $S_1 = \overline{A} \cdot C$
- Grupo verde: $S_2 = A \cdot D$

Produto de somas:

- Grupo amarelo: $P_1 = D + \overline{A}$
- Grupo azul: $P_2 = A + C$

Expressões mínimas para $F(A, B, C, D)$:

$$F(A, B, C, D) = \overline{A} \cdot C + A \cdot D = (D + \overline{A}) \cdot (A + C)$$

Resumo de regularidades

Para uma função de N variáveis:

- Grupos de 2 (= 2^1) células: termos de N-1 variáveis
- Grupos de 4 (= 2^2) células: termos de N-2 variáveis
- Grupos de 8 (= 2^3) células: termos de N-3 variáveis
- **Soma de produtos:** Para um dado grupo de uns no mapa
  - Se a variável varia (0 e 1): Desaparece
  - Se permanece em 1: Entra direta na expressão (produto)
  - Se permanece em 0: Entra negada na expressão (produtos)
- **Produto de somas:** Para um dado grupo de zeros no mapa
  - Se a variável varia (0 e 1): Desaparece
  - Se permanece em 1: Entra negada na expressão (soma)
  - Se permanece em 0: Entra diretamente na expressão (soma)

Funções especificadas incompletamente

- Pode ocorrer que uma função booleana não esteja especificada para todas as combinações das variáveis.
- Exemplo: Função booleana de duas entradas que calcula a diferença entre duas entradas A e B apenas se $A \geq B$.

A	B	D(A,B)
0	0	0
0	1	X
1	0	1
1	1	0

  A especificação assume que o resultado de $0 - 1$ é irrelevante, logo, não importa o seu valor (situação *don't care*).
- X **não** representa um valor diferente de 0 e 1.
- O valor de D(0,1) é irrelevante, mas, a expressão booleana da função produz um resultado específico.
- Escolher esse resultado de forma a minimizar a expressão.

Mapa de Karnaugh de função especificada incompletamente

B \ A	0	1
0	0	X
1	1	0

B \ A	0	1
0	0	0
1	1	0

$$D(A, B) = \overline{A} \cdot B$$

A alternativa seria pior:

B \ A	0	1
0	0	1
1	1	0

$$D(A, B) = \overline{A} \cdot B + A \cdot \overline{B}$$

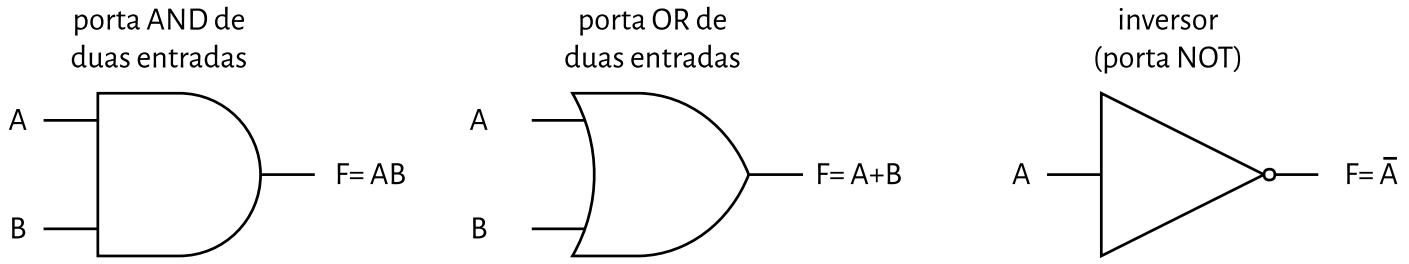
1 Álgebra de Boole
Representação Abstrata do Processamento Binário
Especificação Algébrica
Representações Canónicas
Mapas de Karnaugh

2 Portas Lógicas
Portas Lógicas Elementares
Descrição Hierárquica de Circuitos

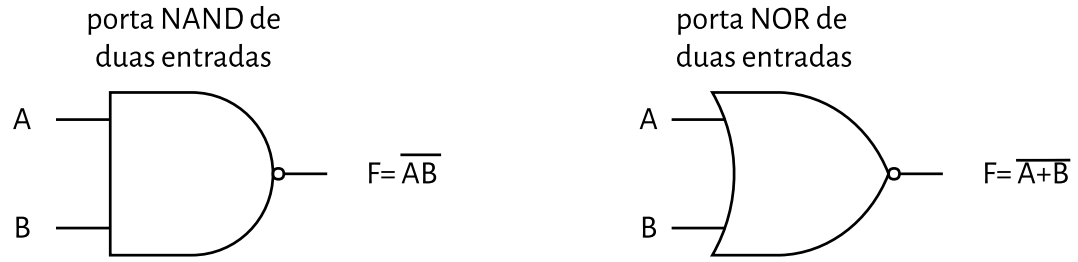
3 Circuitos Padrão
Multiplexadores
Descodificadores Binários
Codificadores

Elementos para Processamento Lógico de Informação

- Para realizar fisicamente o processamento da informação, são usados circuitos eletrónicos que realizam as funções lógicas elementares: **portas lógicas**.
- Quando não interessam os detalhes de implementação, usam-se símbolos para representar cada porta lógica.



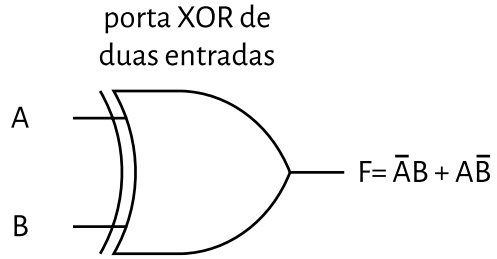
- Também existem portas lógicas que combinam a negação com outras operações lógicas.

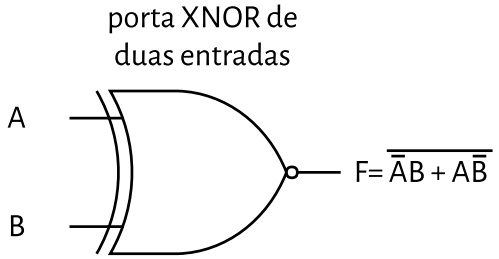


- Existem versões destas portas com mais entradas (3, 4, . . .) [exceto inversor].

As Portas Lógicas XOR e XNOR

- OU-exclusivo: $F = A \oplus B = \overline{A}B + A\overline{B}$
  OU-exclusivo negado: $F = A \odot B = AB + \overline{A}\,\overline{B}$





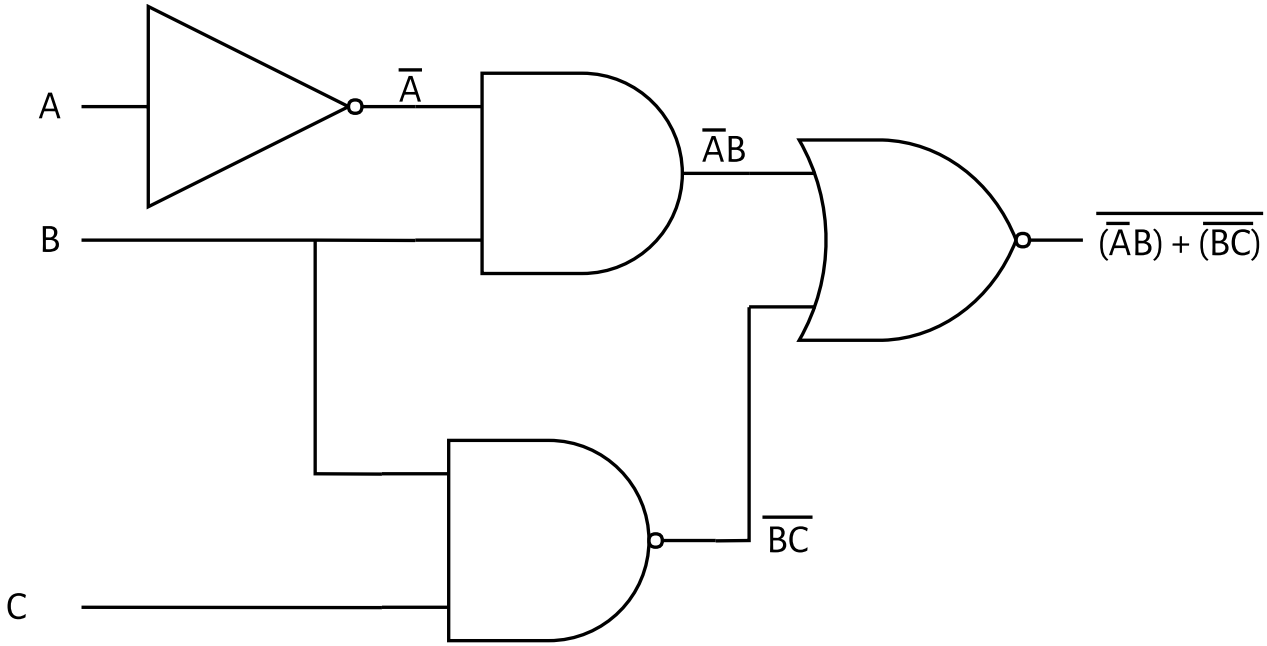
A	B	$A \oplus B$	$A \odot B$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1

- OU-exclusivo é igual a 1 quando as entradas são diferentes.
- OU-exclusivo negado é igual a 1 quando as entradas são iguais.

- $A \oplus (B \oplus C) = (A \oplus B) \oplus C$ $\quad A \odot (B \odot C) = (A \odot B) \odot C$
- $A \oplus 0 = A$ $\quad A \oplus 1 = \overline{A}$ $\quad A \oplus A = 0$ $\quad A \oplus \overline{A} = 1$
- $A \oplus \overline{B} = \overline{A} \oplus B = \overline{(A \oplus B)} = A \odot B$

Circuitos Compostos por Portas Lógicas

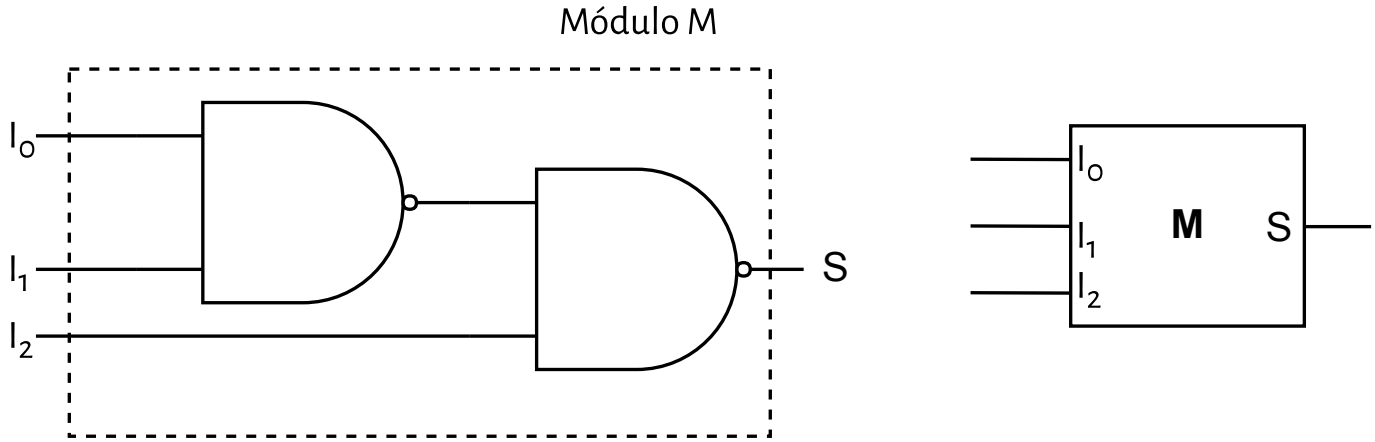
- Existe uma correspondência direta entre uma função lógica e o circuito de portas lógicas elementares que a realiza.
  Exemplo:



- Quanto mais simples for a expressão, mais pequeno é o circuito.
- Circuitos diferentes podem realizar a mesma função lógica (correspondem a expressões equivalentes).

Complexidade e Modularidade

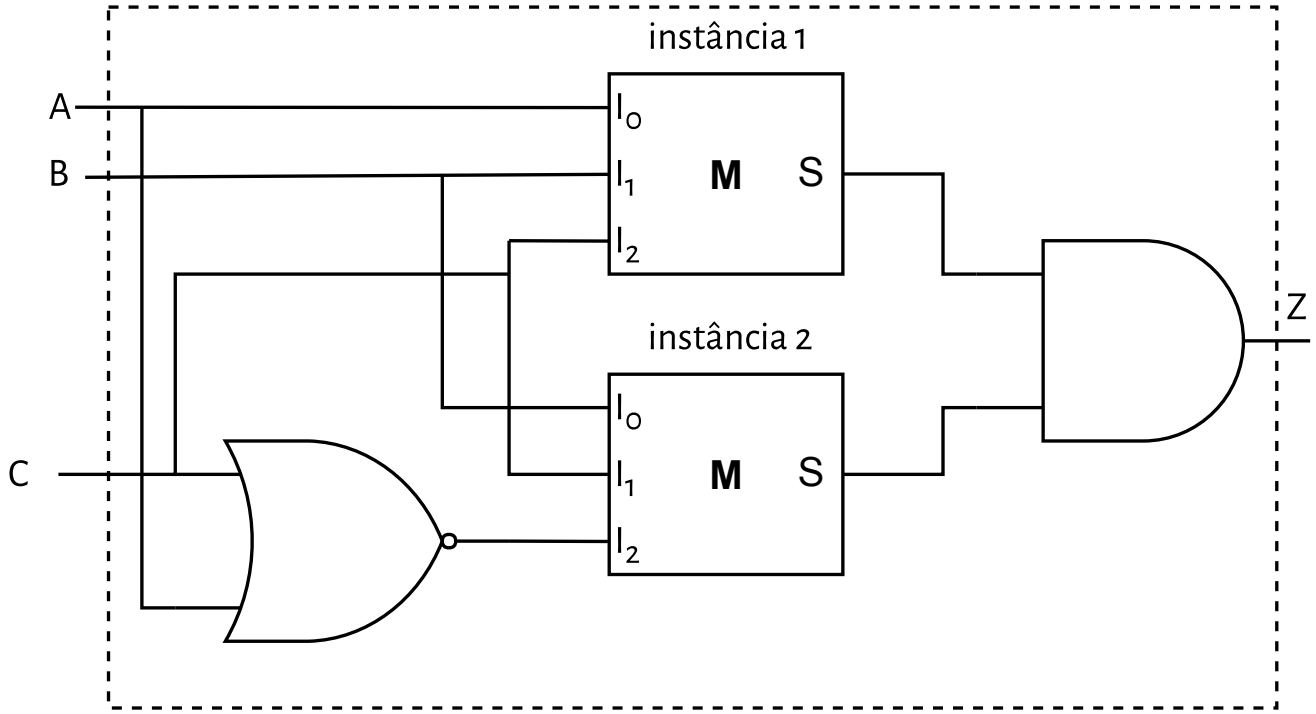
- Circuitos lógicos podem ser muito complexos $\Rightarrow$ como projetá-los?
- Dividir e conquistar: usar uma abordagem modular e hierárquica:
  - Portas lógicas são usadas para descrever funções lógicas mais complexas, implementadas por "módulos".
  - Os módulos pode ser usados na definição de circuitos lógicos mais complexos.



- Função lógica $M(I_2, I_1, I_0) = \overline{\overline{I_0 I_1}\, I_2}$

Descrição Hierárquica

- Módulos podem ser combinados com outros módulos e portas lógicas. Exemplo de módulo hierárquico $Z(A, B, C)$:

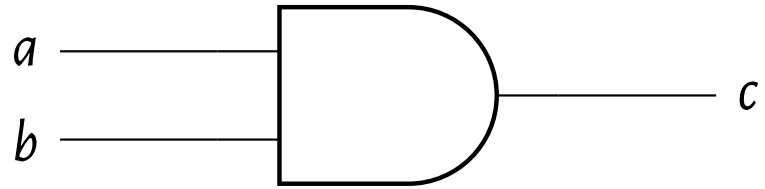


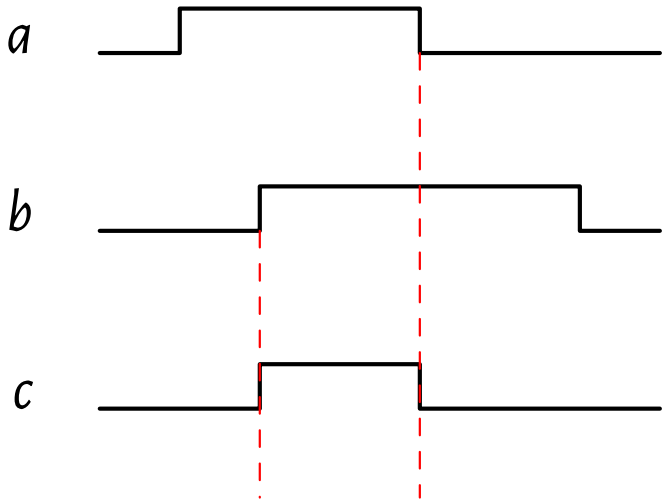
- Este processo pode ser repetido um número arbitrário de vezes.

Tempo de Propagação

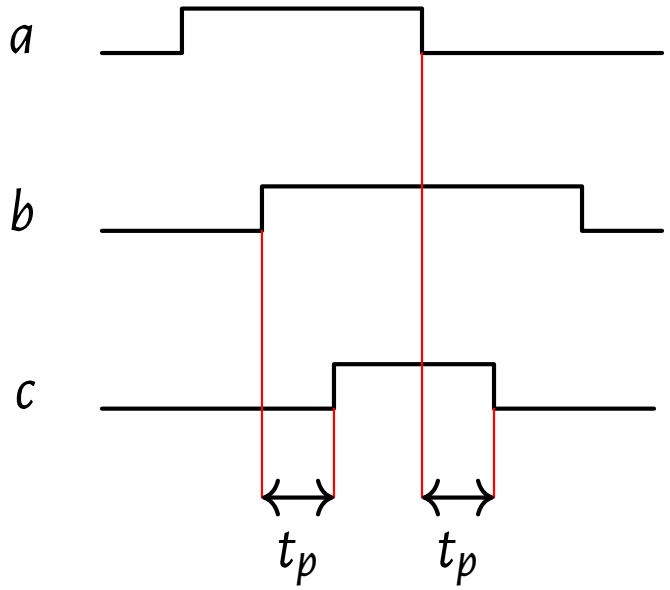
- Em várias situações, é preciso ter em conta as **caraterísticas físicas** das portas lógicas.
- Entre as mais importantes está o **tempo de propagação** (atraso).
  - Uma porta lógica física necessita de algum tempo para reagir a uma alteração da entrada.
  - O tempo de propagação é pequeno, mas não nulo (na ordem dos nanossegundos).
  - Esse tempo depende da tecnologia usada, do número de portas lógicas ligadas à saída, da complexidade da porta lógica e de outros fatores.
- Vamos usar um modelo muito simples: o tempo de propagação (t_p) é **fixo e igual** para alterações de qualquer uma das entradas.
  - Este modelo, embora simples, já permite abordar alguns dos aspetos do comportamento real dos circuitos lógicos.

Efeitos do Tempo de Propagação





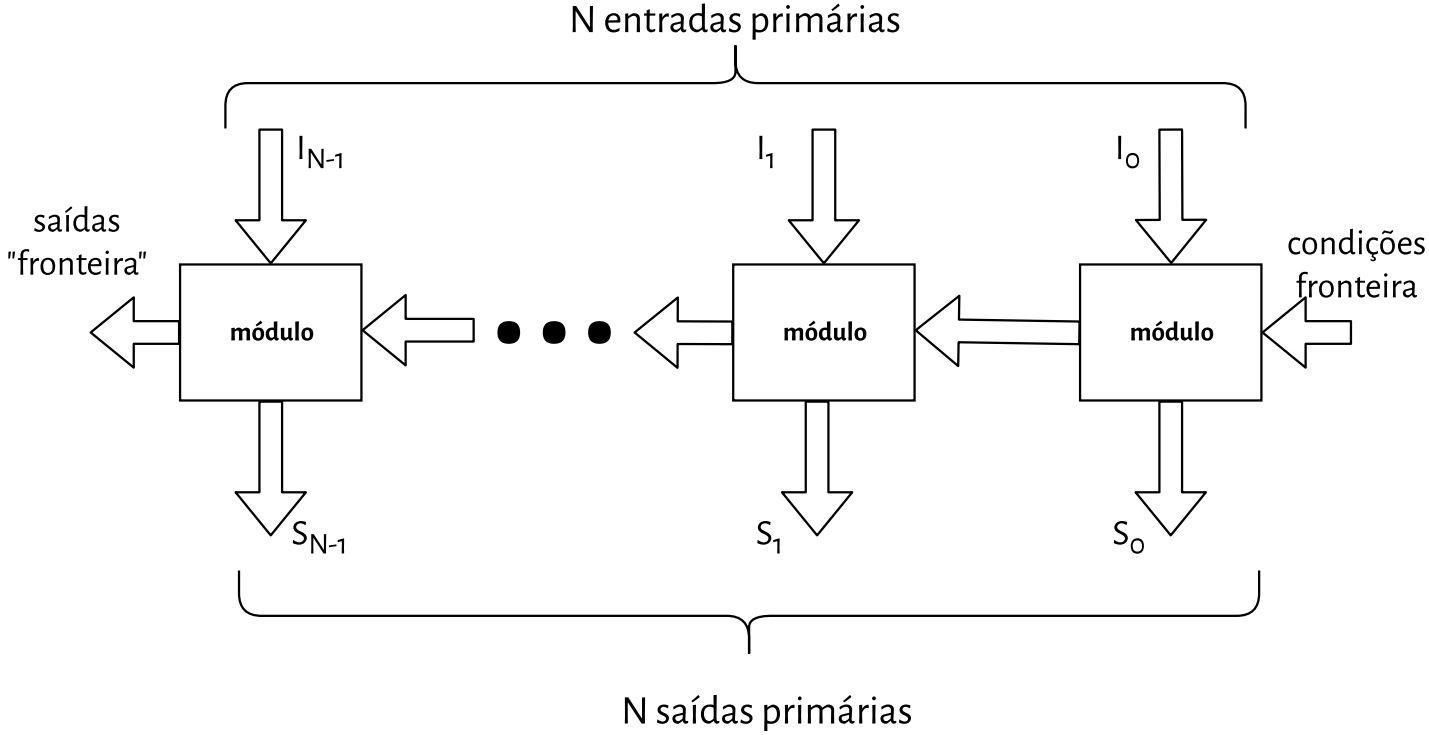
Tempo de propagação nulo



Tempo de propagação $t_p \neq 0$

Circuitos Iterativos

- Circuitos **iterativos** são constituídos por repetições de um mesmo módulo.

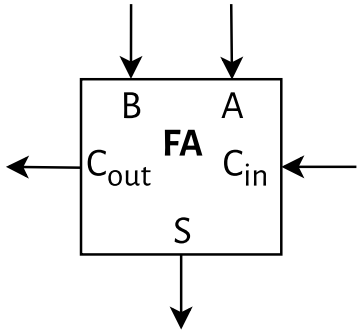


  - Identificar e projetar o módulo de base;
  - Interligar uniformemente instâncias do módulo base (eventualmente usando portas lógicas adicionais).

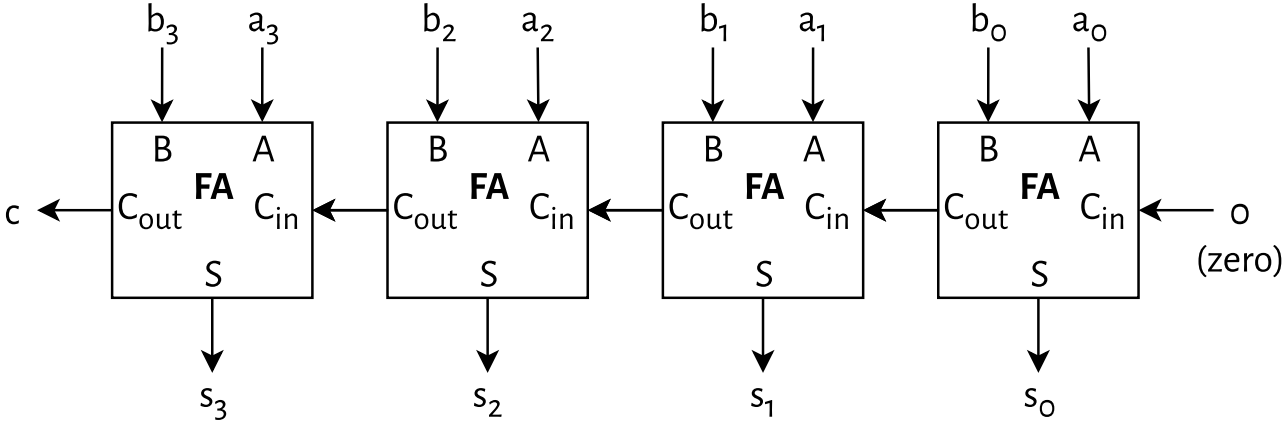
- Este tipo de circuito pode ser facilmente expandido.

Somador do Tipo «Ripple-Carry»

- O somador do tipo «ripple-carry» é um bom exemplo de um circuito iterativo.
- Exemplo: somar dois números de 4 bits $\mathbf{a_3a_2a_1a_0}$ e $\mathbf{b_3b_2b_1b_0}$ usando um módulo que calcula a soma de 2 bits (FA: *full adder*). Resultado: $\mathbf{s_3s_2s_1s_0}$ e $\mathbf{c}$
- Elemento individual:

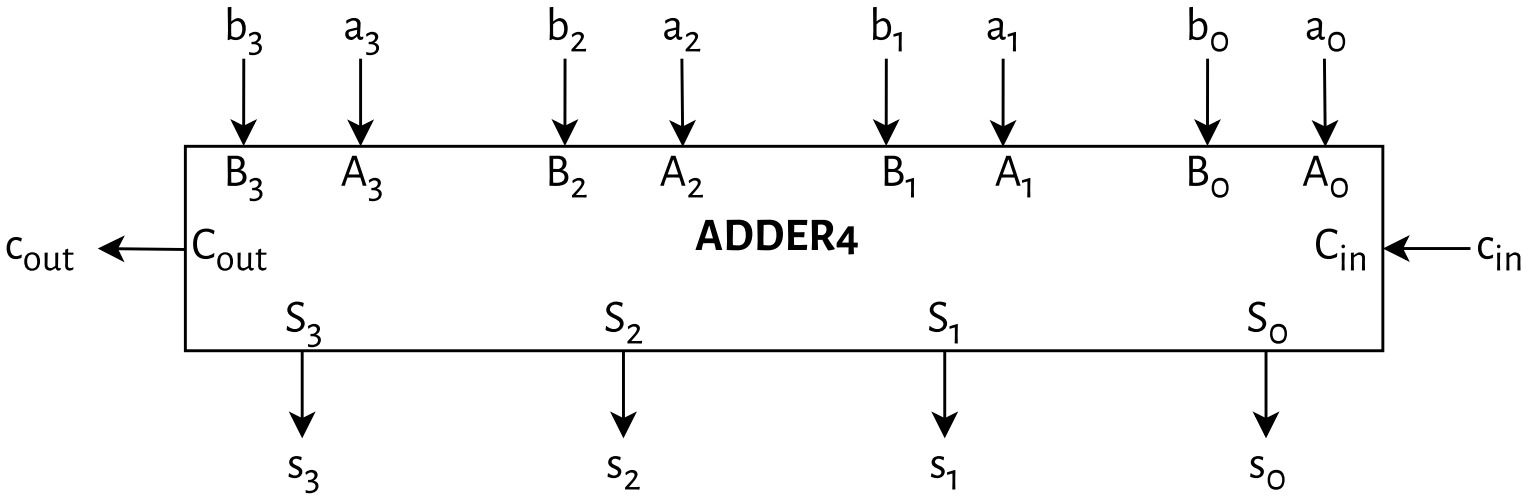


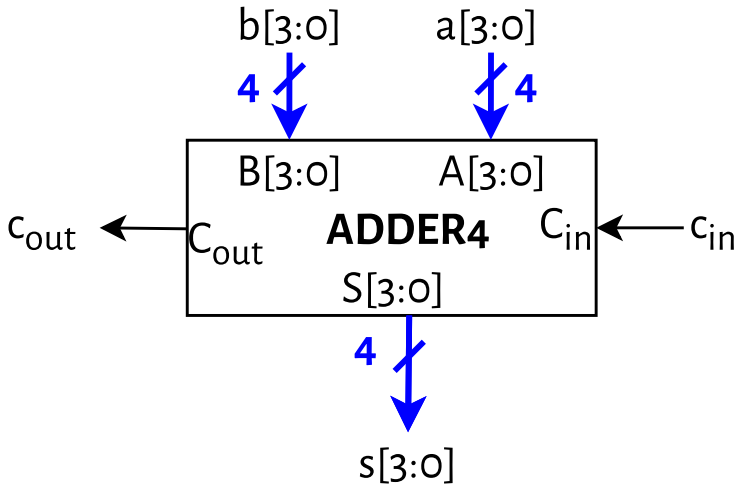
- Somador de operandos de 4 bits:



Simplificar Diagramas Usando «Barramentos»

Um «barramento» é um grupo de sinais que interessa tratar como uma unidade.
O seu uso simplifica muito os diagramas (comparar as duas figuras).





1 Álgebra de Boole
Representação Abstrata do Processamento Binário
Especificação Algébrica
Representações Canónicas
Mapas de Karnaugh

2 Portas Lógicas
Portas Lógicas Elementares
Descrição Hierárquica de Circuitos

3 Circuitos Padrão
Multiplexadores
Descodificadores Binários
Codificadores

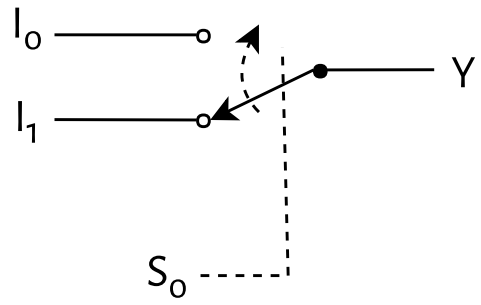
Funções Lógicas Comuns

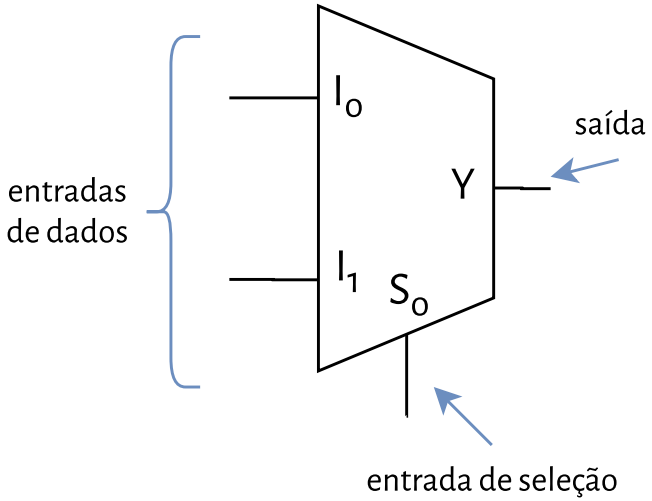
- A experiência mostrou que existe um conjunto de funções lógicas que encontram utilização em muitos sistemas digitais.
- Essas funções são realizadas por **circuitos padrão** de "média complexidade" (i.e., mais complexos que simples portas lógicas).
- A sua utilização facilita o projeto de sistemas digitais.
- O circuito *full adder* pode ser considerado uma dessas funções.
- Outras funções incluem comparadores, des/codificadores de vários tipos, de/multiplexadores.
- Circuitos padrão estão disponíveis no mercado.
- Existem normas para alguns símbolos padrão (ex.: IEEE Graphic Symbols for Logic Functions IEEE-91)

  Mais simples: usar retângulos com entradas (à esquerda) e saídas (à direita).

Multiplexador de 2 Entradas

Um multiplexador (*multiplexer, mux*) 2:1 é um circuito que permite selecionar o valor presente numa de duas entradas de dados.



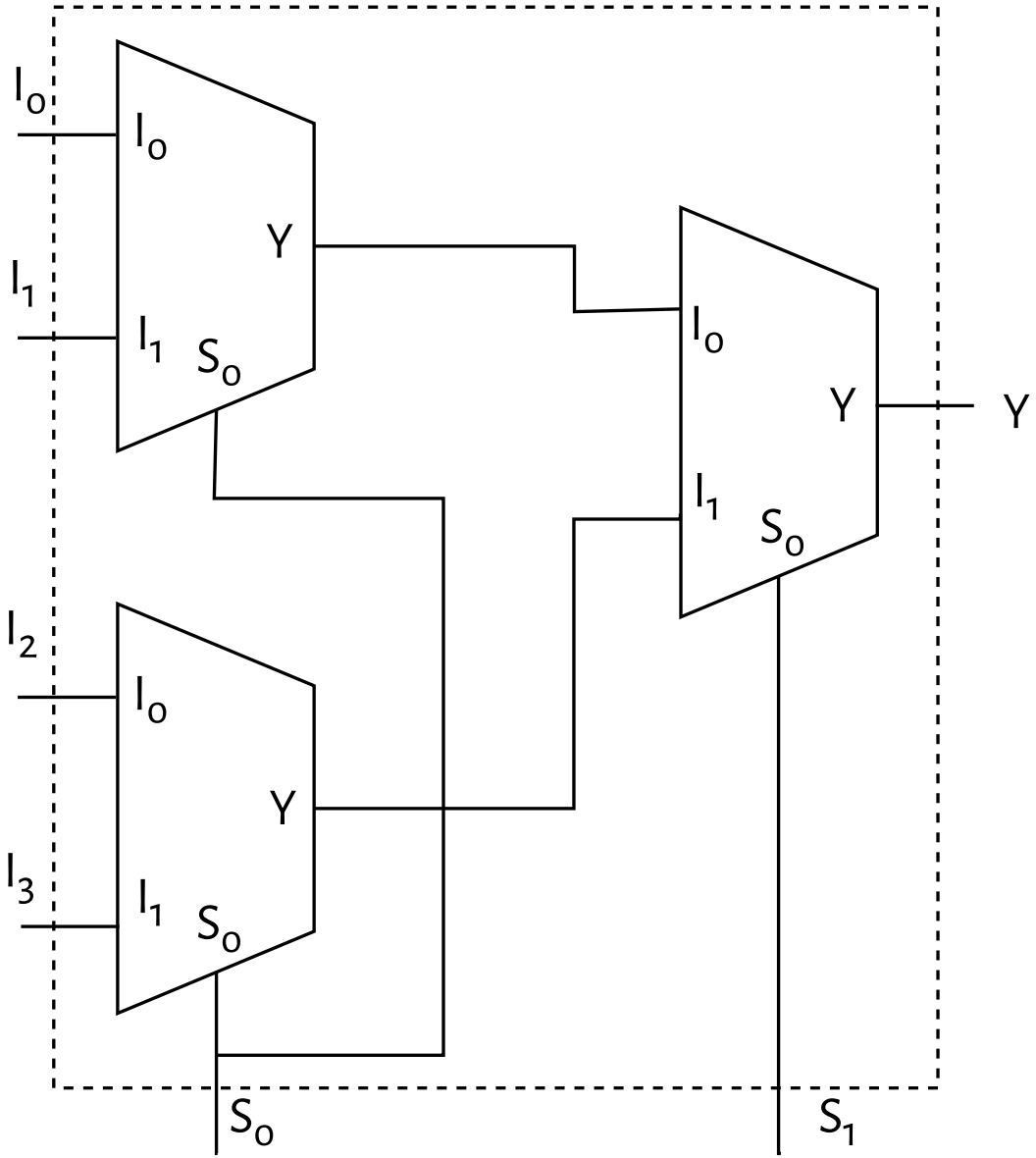


S_0	I_1	I_0	mux(S_0, I_1, I_0)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- Existem multiplexadores 4:1, 8:1, ..., 2^N:1.
- Um multiplexador 2^N:1 tem N entradas de controlo $S_{N-1}, \ldots, S_1, S_0$.
- Todas as expressões lógicas podem ser implementadas com multiplexadores 2:1. (Porquê?)
- Expressões de M variáveis podem ser realizadas com um multiplexador de M entradas de dados.

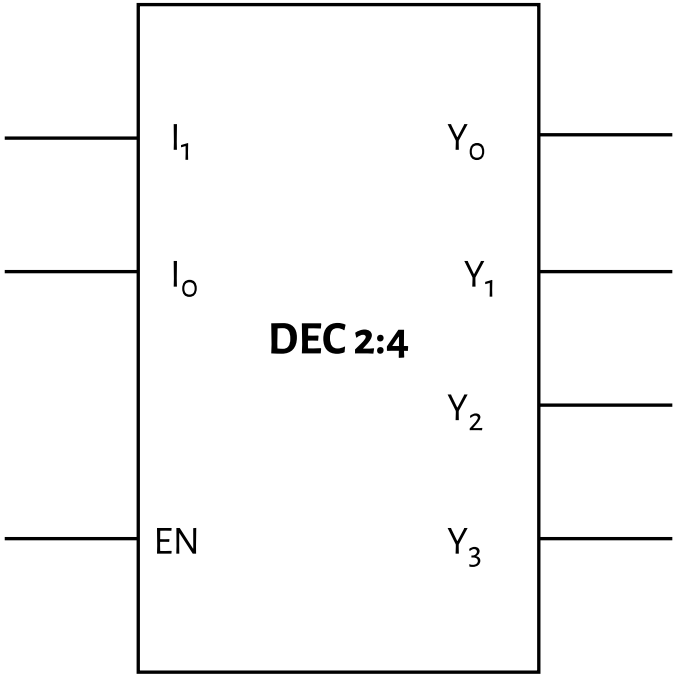
Como Expandir Multiplexadores

Exemplo: construir um multiplexador de 4:1.



Descodificadors Binários

- Descodificador *(decoder)* de N-para-M (geralmente N < M) transforma um código de N bits noutro com mais bits.
- Descodificador binário de N-para-2^N



- Descodificador binário de 2:4

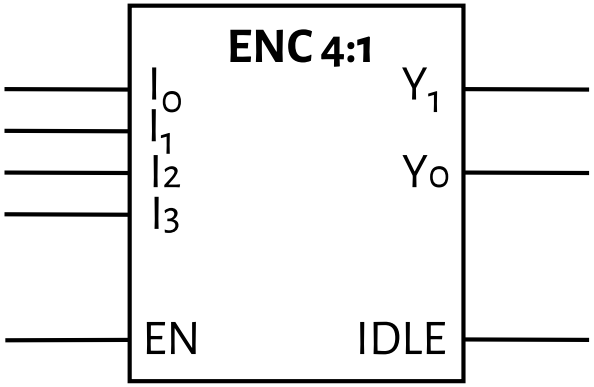
EN	I1	I0	Y3	Y2	Y1	Y0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

- Como fazer um descodificador binário 3:8 ?

- A entrada EN designa-se por **entrada de habilitação** *(enable)*.
- Descodificador binário seguido de porta lógica OU permite realizar todas as funções de N variáveis. (Como?)

Codificador Binário

- Um codificador *(encoder)* transforma um código de X bits num código de Y bits, com X > Y.
- O codificador binário tem 2^N entradas e N saídas (mais sinais de controlo).



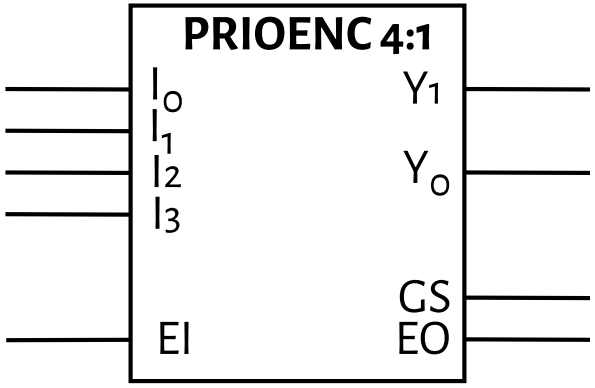
EN	I_3	I_2	I_1	I_0	Y_1	Y_0	IDLE
0	X	X	X	X	0	0	1
1	0	0	0	0	0	0	1
1	0	0	0	1	0	0	0
1	0	0	1	0	0	1	0
1	0	1	0	0	1	0	0
1	1	0	0	0	1	1	0

- Para as restantes combinações de valores de entrada, os valores das saídas não estão definidos!

Codificador de Prioridade

- A saída de um codificador de prioridade é definida pela entrada de maior prioridade que estiver a "1".

Exemplo: codificador de prioridade 4:2 (I_3 tem a maior prioridade; I_0 a menor)



EI	I_3	I_2	I_1	I_0	Y_1	Y_0	GS	EO
0	X	X	X	X	0	0	0	0
1	0	0	0	0	0	0	0	1
1	0	0	0	1	0	0	1	0
1	0	0	1	X	0	1	1	0
1	0	1	X	X	1	0	1	0
1	1	X	X	X	1	1	1	0

EI: *(enable input)* circuito habilitado;
EO: *(enable output)* para habilitar circuito de menor prioridade;
GS: *(got something)* está a "1"se EI=1 e alguma entrada de dados está a "1"

- Desafio: fazer um codificador de prioridade 8:3 com dois codificadores 4:2 e portas lógicas.